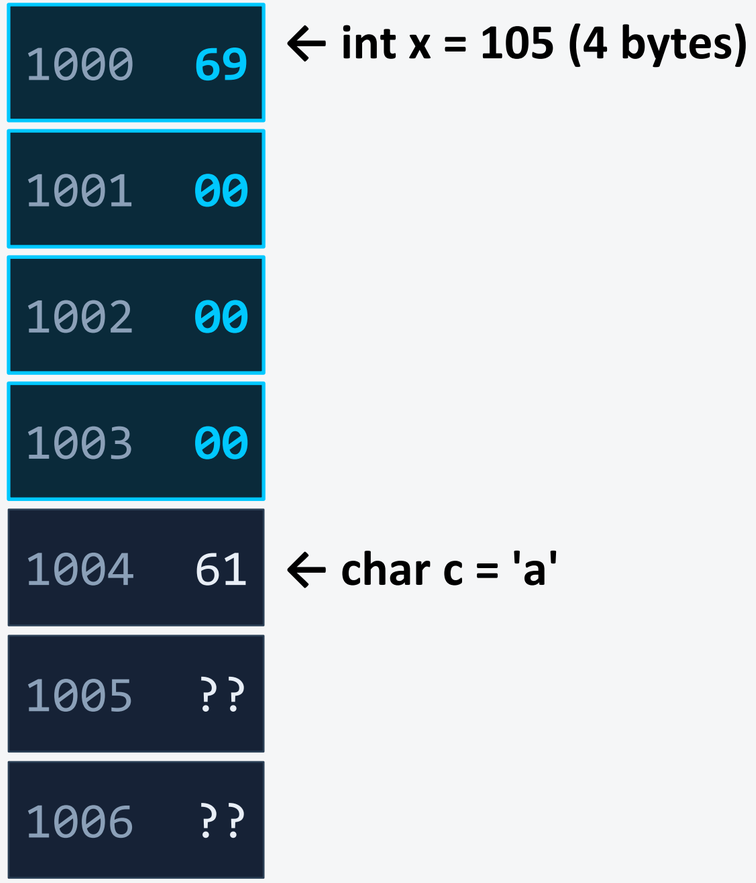
Lecture-10
Pointers

Memory as Blocks of Bytes

- Computer memory is a large array of bytes, each with a unique address.
- Variables are stored in consecutive bytes depending on their type.
- char → 1 byte | int → 4 bytes | double → 8 bytes
- The address of a variable is the address of its FIRST byte.

```
int  x = 105;    // 4 bytes at addr 1000
char c = 'a';    // 1 byte  at addr 1004
```



Memory Diagram

Addresses & Pointers

- A pointer is a variable that stores a memory ADDRESS.
- Declare: int *ptr; ptr holds the address of an int.
- & (address-of) returns the address of a variable.
- * (dereference) accesses the value AT the stored address.
- A null pointer (nullptr) points to nothing.

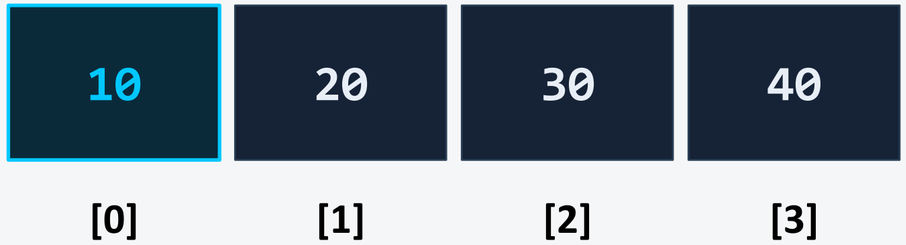
Visualisation


```
int  x   = 42;
int *ptr = &x;          // ptr stores address of x
cout << x;              // 42  (value)
cout << &x;             // e.g. 0x61fe14  (address)
cout << ptr;            // same address as &x
cout << *ptr;           // 42  (dereferenced = value at address)
*ptr = 100;             // changes x through the pointer
cout << x;              // 100
```

Array Name is a Pointer

- An array name evaluates to the address of its FIRST element.
- arr ≡ &arr[0] both give the same address.
- You can use a pointer to traverse an array.
- Subscript notation arr[i] is equivalent to *(arr + i).

Array in Memory




```
int arr[4] = {10, 20, 30, 40};
int *p = arr;           // p points to arr[0]
cout << arr;            // address e.g. 0x61fe00
cout << &arr[0];        // SAME address
cout << p;              // SAME address
cout << *p;             // 10  (arr[0])
cout << *(p+1);         // 20  (arr[1])
cout << arr[2];         // 30  ≡ *(arr+2)
```

```
arr[i]  ≡  *(arr + i)  ≡  *(p + i)
```

Why We Can't Increment an Array Name

arr++ is a COMPILE ERROR:

- An array name is a CONSTANT pointer. Its value is fixed at compile time.
- It always points to the start of the array and cannot be reassigned.
- arr++ would attempt to change that fixed base address → illegal.
- Use a separate pointer variable (int *p = arr;). p++ is perfectly fine.

Analogy

Think of the array name like a building's permanent address plaque on the wall.
You CANNOT peel it off and move it to the next building.
But you CAN hold a copy of the address on a piece of paper (a pointer) and walk to the next building yourself.

```
int arr[4] = {10, 20, 30, 40};
// ❌ ILLEGAL because array name is a constant pointer
arr++;              // error: lvalue required as increment operand
// ✅ LEGAL because p is a regular pointer variable
int *p = arr;
p++;                // p now points to arr[1]
cout << *p;         // 20
```

Pointer Arithmetic & Associativity

- p + n moves n ELEMENTS forward (not n bytes); scaled by sizeof(type).
- p++ / p-- advance/retreat by one element.
- p1 - p2 gives the number of elements between two pointers.
- Comparison operators (<, >, ==) work between pointers of the same array.

```
int arr[5] = {5, 10, 15, 20, 25};
int *p = arr;          // points to arr[0]
p++;                   // p → arr[1], address +=
sizeof(int) = 4
p += 2;                // p → arr[3]
cout << *p;            // 20
// ── Associativity of * and ++ ──────────────────────
cout << *p++; // prints *p THEN increments p   (postfix)
cout << *++p; // increments p FIRST, then prints *p
cout << (*p)++;// increments the VALUE at p, not the
pointer
```

Operator Precedence (* vs ++)

Expression	Meaning
*p++	*(p++) — use *p, then p++
*++p	*(++p) — increment p, then use *p
++*p	++(*p) — increment value at p
(*p)++	(*p)++ — value at p, then inc value

Quick Check 🧠 Pointers & Memory

Q1. What does `*ptr++` do?

A Increments the value at ptr, then moves ptr	**B** Uses the value at ptr, then advances ptr to next element
C Moves ptr back by one element	**D** Compile error — cannot mix * and ++

✓*Answer: B*

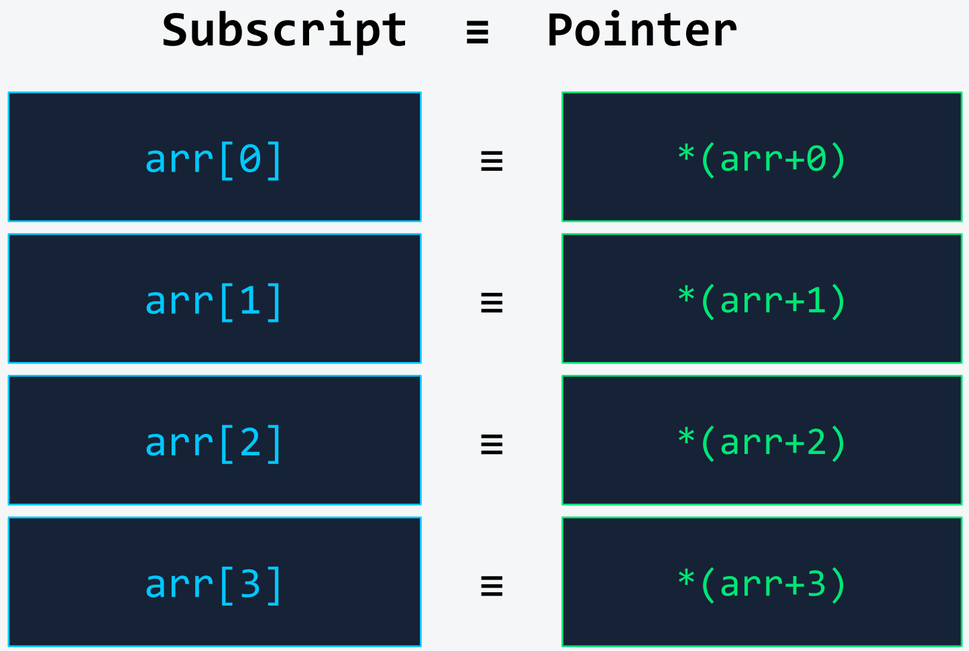
Q2. Given `int arr[3];` which statement is ILLEGAL?

A int *p = arr;	**B** p++;
C arr++;	**D** *(arr + 2) = 5;

✓*Answer: C (array name is a constant pointer.)*

Pointers and Arrays

- Pointer traversal is equivalent to array subscript traversal.
- Passing an array to a function passes the base address (pointer).
- Changes inside the function affect the ORIGINAL array.
- Use pointer notation for efficient scanning of large arrays.



Pointers and Arrays

```
// Two ways to print an array
void printArr(int *p, int n) {
    // Method 1 – subscript
    for (int i = 0; i < n; i++)
        cout << p[i] << " ";

    // Method 2 – pointer arithmetic
    for (int *end = p + n; p < end; p++)
        cout << *p << " ";
}

int main() {
    int nums[] = {3, 6, 9, 12};
    printArr(nums, 4);   // passes &nums[0]
}
```

Subscript	≡	Pointer
arr[0]	≡	*(arr+0)
arr[1]	≡	*(arr+1)
arr[2]	≡	*(arr+2)
arr[3]	≡	*(arr+3)

Pointers and Functions

- Pass by pointer: pass the address → function can modify the original.
- Returning a pointer: return a dynamically allocated value (avoid returning local vars!).
- Pointer to function: stores the address of a function for callbacks.

Pass by Value vs Pass by Pointer

By Value

A COPY is made.
Original unchanged.

By Pointer

Address is passed.
Original IS modified.

Pointers and Functions

```
// ── Pass by pointer ──────────────────────────
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}
// ── Return pointer (dynamic) ─────────────────
int* createValue(int v) {
    int *p = new int(v);  // heap — safe to return
    return p;
}
// ── Pointer to function ──────────────────────
int add(int a, int b) { return a + b; }
int main() {
    int x = 3, y = 7;
    swap(&x, &y);         // x=7, y=3

    int (*fn)(int,int) = add;
    cout << fn(2, 5);     // 7
}
```

Pass by Value vs Pass by Pointer

By Value

A COPY is made.
Original unchanged.

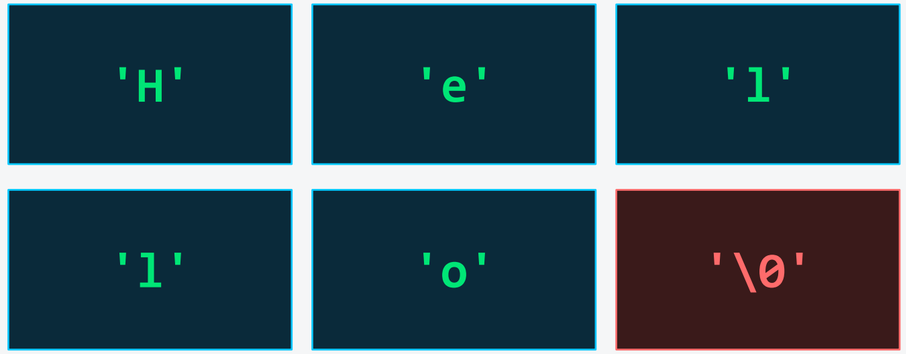
By Pointer

Address is passed.
Original is modified.

Pointers and C-Type Strings

- A C-string is a char array terminated by '\0' (null character).
- A char* pointer to a string literal points to the first character.
- Traverse with a pointer until '\0' is found.
- Standard C-string functions: strlen, strcpy, strcat, strcmp.
- ⚠ String literals are read-only. Never modify via pointer to literal.

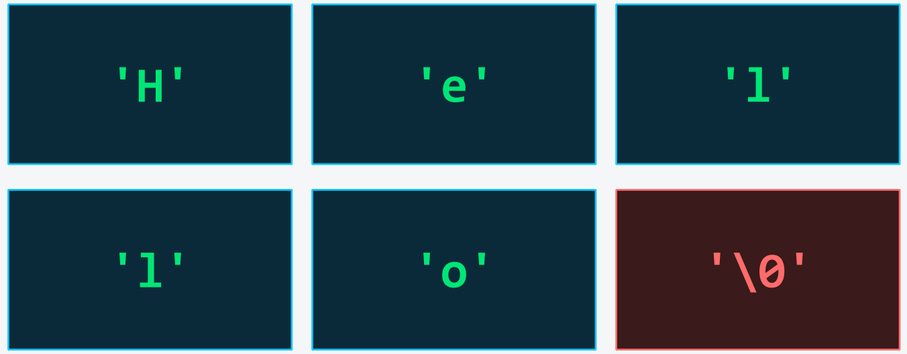
Memory Layout



Pointers and C-Type Strings

```
char msg[] = "Hello";   // {H,e,l,l,o,\0}
char *p    = msg;
// Traverse using pointer
while (*p != '\0') {
    cout << *p;
    p++;
}
// Pointer to string literal (read-only!)
const char *lit = "World";  // use const
// Count length manually
int len = 0;
for (char *q = msg; *q != '\0'; q++)
    len++;
cout << len;            // 5
```

Memory Layout



Memory Management: new and delete

- Stack memory: automatic and freed when scope ends.
- Heap memory: manual and must be freed explicitly.
- new allocates on the heap and returns a pointer.
- delete frees a single heap object. delete[] frees a heap-allocated array.

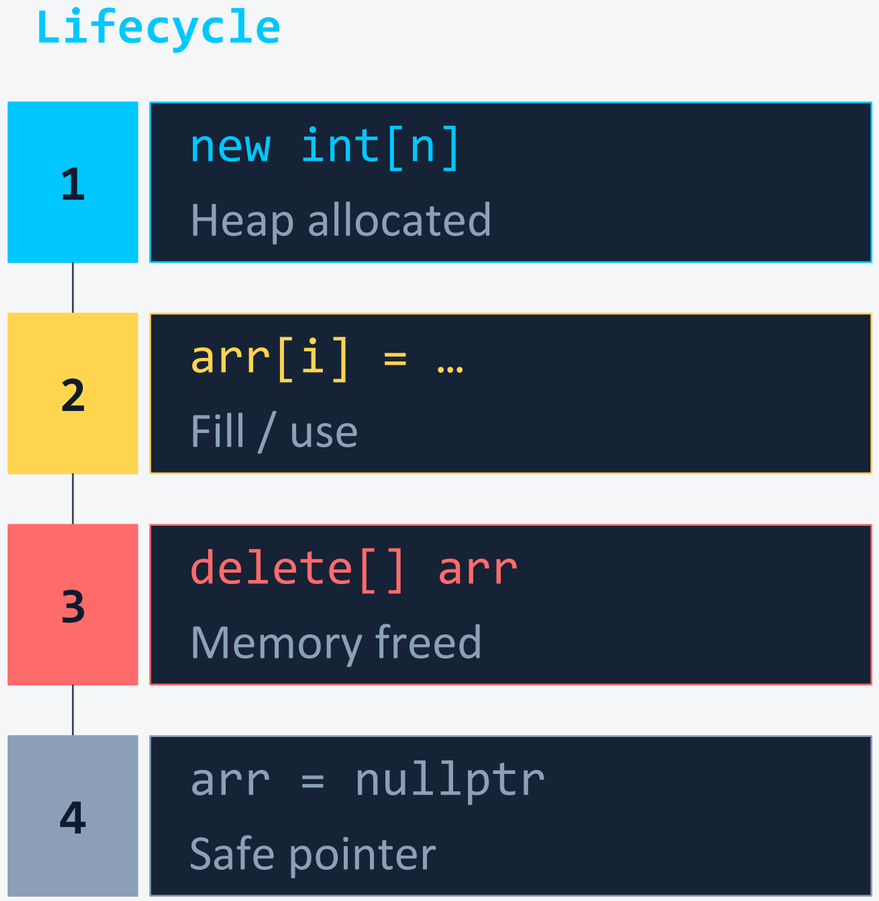
```
// Allocate a single int
int *p = new int(42);
cout << *p;              // 42
delete p;                // free it
p = nullptr;             // good practice
// Allocate an array of 5 ints
int *arr = new int[5];
arr[0] = 10; arr[1] = 20;   // ...fill...
delete[] arr;    // MUST use delete[] for arrays
```

Stack vs Heap

STACK	HEAP
• local vars	• new / delete
• auto freed	• manual management
• fast	• large allocs
• limited	• slower

Creating an int Array with new

- Use new int[n] when the size is known only at runtime.
- The returned pointer points to the FIRST element.
- Always free with delete[].



Creating an int Array with new

```
#include <iostream>
using namespace std;
int main() {
    int n;
    cout << "Enter size: ";
    cin  >> n;
     int *arr = new int[n];
     for (int i = 0; i < n; i++)
         arr[i] = (i + 1) * 10;
     for (int i = 0; i < n; i++)
         cout << arr[i] << " ";
     delete[] arr;
    arr = nullptr;  // ← prevents dangling
pointer
    return 0;
}
```

Lifecycle

1. `new int[n]` — Heap allocated
2. `arr[i] = …` — Fill / use
3. `delete[] arr` — Memory freed
4. `arr = nullptr` — Safe pointer

Calculating Bytes for a Heap int Array

- sizeof(int) gives the size of one int in bytes (usually 4).
- Total bytes = n × sizeof(int)
- sizeof() is evaluated at COMPILE TIME.
- You can also use sizeof on an existing stack array to get total bytes.

Formula

Total Bytes

= n × sizeof(int)

⚠ sizeof Pitfall

sizeof(ptr) returns the size of the POINTER (4 or 8), not the array it points to!
Always track n separately.

Calculating Bytes for a Heap int Array

```
int n = 10;
int *arr = new int[n];
// Total bytes occupied on heap
int bytes = n * sizeof(int);
cout << "Bytes: " << bytes;     // Bytes: 40
int stack[10];
cout << sizeof(stack);          // 40  (total
array bytes)
cout << sizeof(arr);            // 8   (just
pointer size! 64-bit)
// sizeof(arr)/sizeof(arr[0]) does NOT work for
heap arrays

int count = bytes / sizeof(int);  // 40 / 4 = 10
```

Formula

Total Bytes

= n × sizeof(int)

⚠ sizeof Pitfall

sizeof(ptr) returns the size of the POINTER (4 or 8), not the array it points to!
Always track n separately.

Quick Check 🧠 Memory Management

Q3. Which statement correctly frees a heap int array pointed to by arr?

A	delete arr;	**B**	free(arr);
C	delete[] arr;	**D**	arr = nullptr;

✓*Answer: C*

Q4. int *p = new int[6]; How many bytes are allocated?

A	6 bytes	**B**	24 bytes
C	8 bytes (pointer size)	**D**	48 bytes

✓*Answer: B*

Pointers to Objects

- A pointer can store the address of any object (class instance).
- Use -> to access members through a pointer (arrow operator).
- Heap objects: Dog *d = new Dog("Rex"); … delete d;
- ptr->member is shorthand for (*ptr).member
- Essential for dynamic object arrays and polymorphism.

Arrow Operator ->

ptr->member is EXACTLY the same as (*ptr).member

Stack vs Heap Object

Stack Dog rex(…)

auto-destroyed

Heap new Dog(…)

must delete manually

Pointers to Objects

```
class Dog {
public:
    string name;
    int    age;
    Dog(string n, int a) : name(n), age(a) {}
    void bark() {cout << name << " says: Woof!
"; } };
int main() {
    Dog  rex("Rex", 3);        // stack object
    Dog *p  = &rex;   // pointer to stack object
    p->bark();                 // Rex says: Woof!
    cout << p->name;           // Rex
    Dog *h = new Dog("Max", 5); // heap object
    h->bark();
    cout << (*h).age;    // 5  (same as h->age)
    delete h;
}
```

Arrow Operator ->

ptr->member is EXACTLY the same as (*ptr).member

Stack vs Heap Object

Stack Dog rex(…)

auto-destroyed

Heap new Dog(…)

must delete manually

Pointers to Pointers

- int **pp is a pointer to a pointer-to-int.
- Useful for sorting arrays of objects WITHOUT copying objects.
- Build an array of pointers to objects, then sort the pointers.
- Dereferencing: **pp accesses the actual value.

Why pointer-to-pointer?

```
int x = 5;

int *p = &x;

int **pp = &p;

**pp = 99;  // x is now 99
```

Sorting only swaps the pointers. The actual Student objects stay in memory. Very efficient for large objects!

Pointers to Pointers

```
struct Student { string name; int gpa; };
// Sort pointers to students by GPA
void sortByGPA(Student **arr, int n) {
    for (int i = 0; i < n-1; i++)
      for (int j = 0; j < n-i-1; j++)
        if (arr[j]->gpa < arr[j+1]->gpa) {
            Student *tmp = arr[j];  // swap pointers only
            arr[j]   = arr[j+1];
            arr[j+1] = tmp; }}
int main() {
    Student s1{"Alice", 90}, s2{"Bob", 75}, s3{"Cara", 88};
    Student *ptrs[3] = {&s1, &s2, &s3};
    sortByGPA(ptrs, 3);
    for (int i = 0; i < 3; i++)
        cout << ptrs[i]->name << " " << ptrs[i]->gpa << “ ";
    // Output: Alice 90 / Cara 88 / Bob 75
}
```

Quick Check 🧠 Objects & Pointer-to-Pointer

Q5. Given `Dog *d = new Dog("Buddy");` how do you access the name?

A d.name	**B** d->name OR (*d).name
C *d.name	**D** &d->name

✓*Answer: B use -> or (*ptr).member for pointer to object.*

Q6. When sorting with `Student **arr`, what gets swapped?

A The Student objects (copied in memory)	**B** Only the pointer values in the arr array
C The GPA values inside each Student	**D** Nothing — pointers cannot be swapped

✓*Answer: B only pointers swap; original objects stay put.*

Debugging Pointers

Dangling Pointer

Pointer to memory that has been freed.
Accessing it is undefined behaviour.

Fix: Set to nullptr after delete.

Null Dereference

Dereferencing a nullptr causes
a segmentation fault (crash).

Fix: Always check ptr != nullptr before use.

Memory Leak

Heap memory never freed → program
consumes RAM until it crashes.

Fix: Every new must have a matching delete.

Buffer Overrun

Writing past the end of an array.
Corrupts adjacent memory.

Fix: Track size; never exceed bounds.

Debugging Pointers

```
// ✅ Safe pointer pattern
int *p = new int(10);
// ... use p ...
delete p;
p = nullptr;                    // prevents dangling
if (p != nullptr) *p = 5; // guard before dereference
```

Quick Check 🧠 Debugging & Best Practices

Q7. What is a dangling pointer?

A	A pointer that has never been initialized	**B**	A pointer to memory that has been freed
C	A pointer that equals nullptr	**D**	A pointer declared inside a function

✓*Answer: B — set pointer to nullptr after delete to avoid this.*

Q8. Which is the correct way to check before dereferencing?

A	if (*ptr) *ptr = 5;	**B**	if (ptr != nullptr) *ptr = 5;
C	if (&ptr) *ptr = 5;	**D**	if (ptr == 0) *ptr = 5;

✓*Answer: B — always guard against null before dereferencing.*